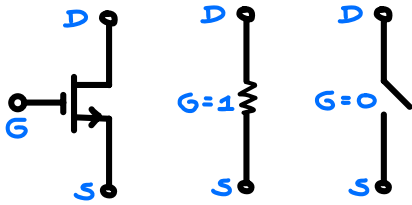


INVERTITORE CMOS

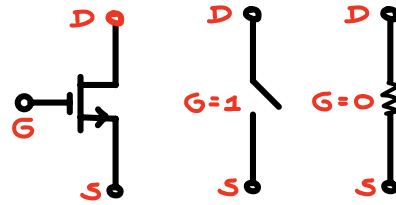
Considero NMOS e PMOS

NMOS



- Quando $V_g = V_{DD}$ il gate è chiuso ed equivale ad una resistenza $R_{on} < 1 \Omega$. Se invece $V_g = 0$ ho una resistenza alta $R_{off} \sim T \Omega$ e non passa nulla.

PMOS



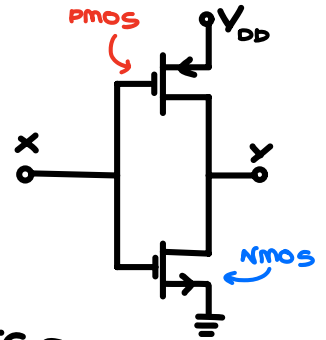
- Funziona esattamente al contrario

Quindi se il circuito è chiuso passa sempre corrente e dissipò un sacco di energia.

Per sistemare uso **invertitore**

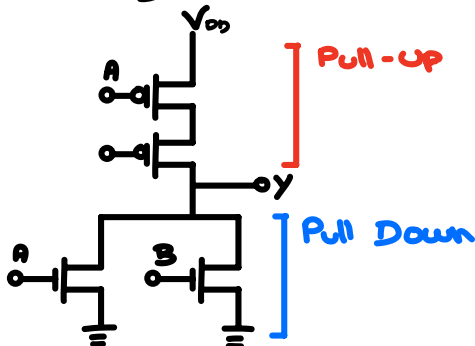
- Rappresenta la funzione $Y = \bar{X}$
- Se $X = 0$ allora il PMOS è acceso, NMOS spento, quindi $Y = V_{DD}$ ovvero 1 logico
- Se $X = 1$ viceversa
- NMOS è detto **pull down network** perché è lui che porta il segnale a 0
- PMOS è detto **pull up network** per il contrario

Posso costruire circuiti con questa logica pull up/down



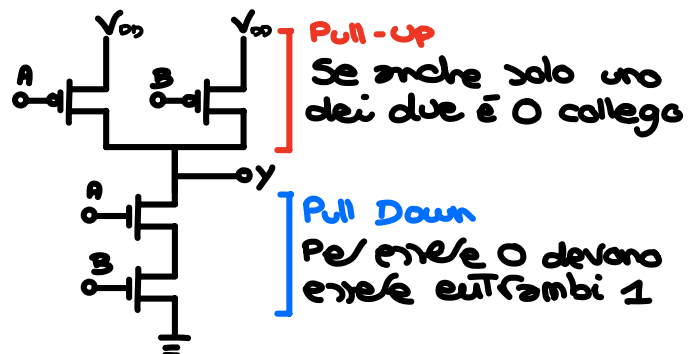
NOR GATE

$$Y = \overline{A+B} = \bar{A}\bar{B}$$



NAND GATE

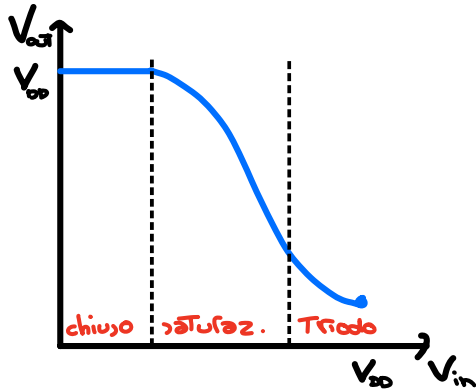
$$Y = \overline{AB} = \bar{A} + \bar{B}$$



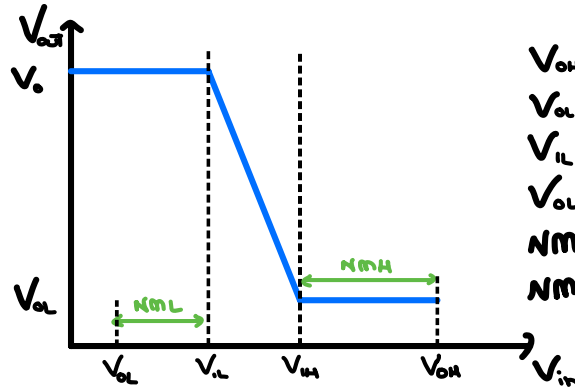
Vediamo le **caratteristiche dell'invertitore**

- ↳ La **voltage Transfer characteristic** descrive il comportamento della tensione in uscita in funzione di quella in ingresso

VTC MOSFET



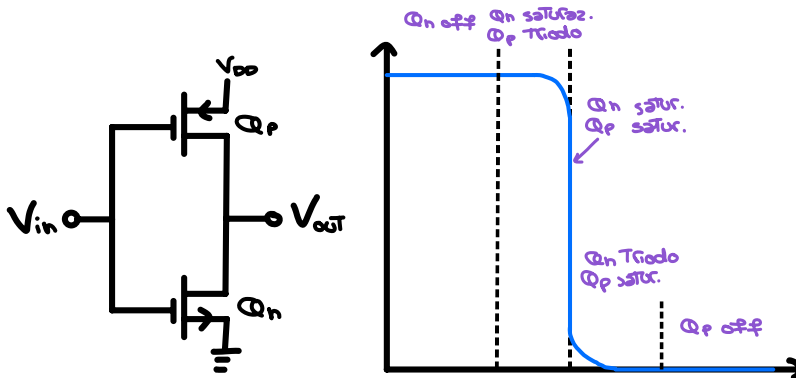
VTC INVERTITORE



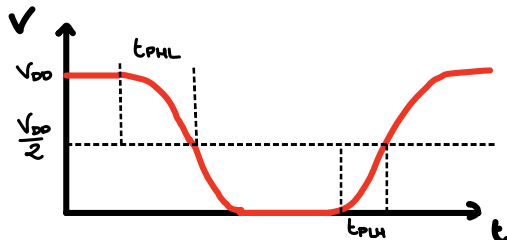
V_{oh} output high level
 V_{ol} output low level
 V_{il} } inversione
 V_{ol} }
 NML noise margin low
 NMH noise margin high

- Nel range NML l'output è sempre V_{ol} , indipendentemente dall'input. Uguali per NMH.
- V_{il} e V_{ih} delimitano la **regione di Transizione**.
- ha **non sensibilità** nei noise margins e la forza dei circuiti digitali.
- Per invertitore ideale la Transizione verticale a $\frac{V_{DD}}{2}$

Vediamo la VTC completa



Il Tempo di propagazione quando un input cambia non è istantaneo, c'è un ritardo



- Ci sono due Tempi di propagazione ($L \rightarrow H$ e $H \rightarrow L$). Non necessariamente uguali
- Definisco **ritardo di propagazione** la media $t_p = \frac{1}{2}(t_{PLH} + t_{PHL})$
- Definisco **massima freq. di switching** $f_m = \frac{1}{2t_p}$

Il ritardo è dovuto a carica/scarica delle capacità presenti nei MOSFET, nei cavi e nei gate.

mandare segnali a frequenza $f > f_m$ non ha senso! l'invertitore non le rileva

RAPPRESENTAZIONI NUMERICHE

Con 3 bit posso rappresentare 2^3 parole diverse

- per i numeri ordinari ho solita scrittura binaria
- per numeri con segno si usa complemento a 2

Esempio con $B=4$

Uso la prima metà normalmente

$$[0000] = 0_d$$

$$[0011] = 3_d$$

...

Dopo 8_d sfrutto circolarità mod B ovvero

$$-1_d = -1_d + 16_d = 15_d = [1111]_b$$

somma 16 e scrivo poi il numero positivo in binario

- ↳ Con 4 bit posso scrivere 8 positivi e 8 negativi
- ↳ se la cifra più a sx in binario è 0 → positivo, 1 → negativo
- ↳ lo 0 viene considerato tra i positivi

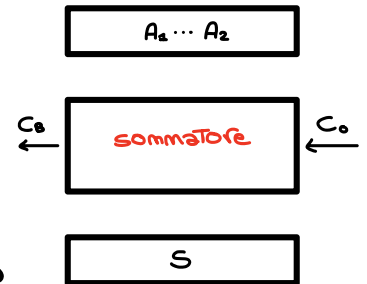
la somma in complemento a 2 si riduce a quella binaria

- ↳ se operandi concordi anche il risultato deve esserlo → overflow
- ↳ $2^n - A_d = \bar{A}_b + 1_b$
- ↳ calcolo sottrazione come $A_{1,b} - A_{2,b} = \text{NOT}(A_{2,b}) + 1_b$

SOMMATORE

Vediamo quindi come implementare effettivamente la somma
Il sommatore ha

- In ingresso $2B+1$ bit
- In uscita $B+1$
- Uno degli ingressi è il carry C_0 che porta il termine unitario in più che serve per la sottrazione



Questo è detto **ripple carry-adder**. Ne esistono altri più veloci che però occupano più spazio circuitale. Nel RCA il numero di porte cresce linearmente con i bit, per altri metodi non è detto.

Full Adder

Vediamo la Tabella di verità dell'elemento base: il full adder

- **simmetria**, scambiando ordine ingressi il risultato non cambia
- **inversione**, negando ingressi si ottiene risultato negato

In genere in base agli ingressi il carry può essere in 3 stati

AB \ C	0	1	
00	0	0	K
01	0	1	P
11	1	1	G
10	0	1	P

- **Kill**; $C_i = 0$ indipendentemente da C_{i-1} , perché $AB = 00$
 - ↳ $K = \bar{A} \cdot \bar{B}$

- **Propagate**; il valore del carry in inglese si conserva
 $P = A \text{ XOR } B$
- **Generate**; $C_i = 1$ indipendentemente da C_{i-1} , perché $AB = 11$
 $G = AB$

Attraverso questi segnali posso esprimere $\begin{cases} S = G + (P \cdot C) \\ C = P \text{ XOR } C \end{cases}$

Ripple Carry Adder

Il sommatore RCA a B bit è dato da B sommatori full adder.

- Tempo di calcolo cresce linearmente con numero di bit (**inaccettabile**)
- dimensione circuito cresce linearmente con numero di bit (**buono**)

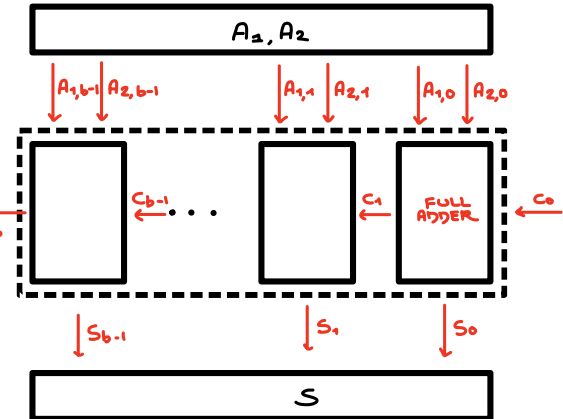
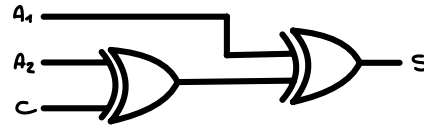
Il full adder fa somma ad 1 bit e C rappresenta il **riporto** dell'operazione.

$$\begin{aligned} S &= \bar{A}_1 \bar{A}_2 C + \bar{A}_1 A_2 \bar{C} + A_1 \bar{A}_2 \bar{C} + A_1 A_2 C \\ &= \bar{A}_1 (A_2 \text{ XOR } C) + A_1 (A_2 \text{ XOR } C)' \\ &= A_1 \text{ XOR } A_2 \text{ XOR } C \end{aligned}$$

oppure posso semplificare aggiungendo $+2A_1 A_2 C$

$$S = A_2 C + A_1 C + A_1 A_2$$

Quindi il circuito può essere semplificato così:



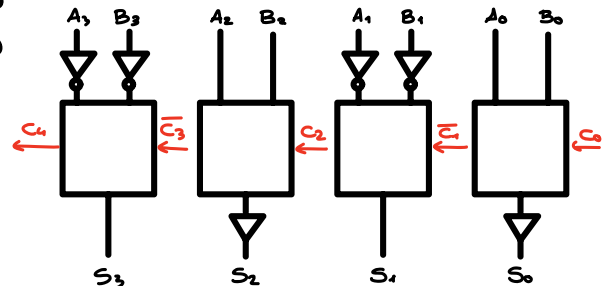
A_1	A_2	C_0	S	C_1
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Ripple Carry Adder Invertente

Il **percorso critico** è costituito dalla **cadena di riporto**, ed è lineare nel numero di bit, massimo quando ho sempre il riporto. Un primo miglioramento è **eliminare l'inverter finale** sul carry, sfruttando la proprietà di inversione.

Il circuito del full adder senza questo invertitore fornisce risultati ogni volta negati.

- ↳ Tollo N invertitori dal percorso critico risparmiando spazio e migliorando performance del 15%



Manchester Carry Chain

Sfrutto lo stato di propagate del carry per velocizzare il sommatore
 $C_{i+1} = G_i + P_i \cdot C_i$

Si fa utilizzando un **pass transistor** che decide o no se il carry viene applicato o no sulla base del segnale P .

- ↳ Pongo **prima** tutti i carry = 1 e poi sulla base di P scelgo se aggiungerli o no.
- ↳ Posso farlo subito perché $P = A \text{ XOR } B$

Quindi G e P sono calcolati in parallelo. C_0 entra nel circuito e si propaga finché $P=1$. Poi se trova $G=1$ un nuovo carry entra nel circuito, ma tutto avviene in un colpo solo.

↳ Il carry si propaga come segnale, non viene ricalcolato ogni volta. Molto più veloce, per più esteso
(4 Transistor per bit + xor per ogni somma)

Carry Skip Adder

Funziona come il ripple, ma utilizza propagate per saltare dei blocchi. Divido i bit in blocchi e alla fine di ogni blocco metto **multiplexer**

↳ Interruttore con + ingressi che ne fa passare solo uno. Se il blocco è tutto propagate porto il carry fino alla fine. Ha un ritardo uguale a quello del ripple carry ma con costante minore, è sempre **lineare in N**.

Carry Select Adder

Parallelizzo il calcolo: un blocco calcola come sarebbe se avessi $C=0$ e l'altro se avessi $C=1$. Quando arriva il vero carry un multiplexer sceglie quale blocco usare, ma è tutto già pronto quindi è più veloce.

Carry Lookahead Adder

Esprime tutti i carry in funzione di G, P e C_0 , in modo da calcolarli tutti subito.

$$C_1 = G_0 + P_0 C_0$$

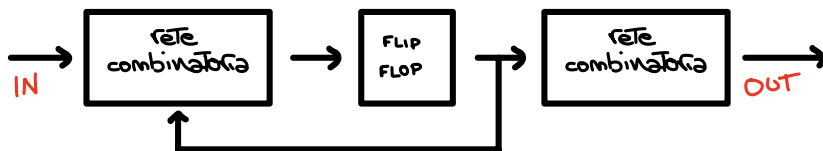
$$C_2 = G_1 + P_1 C_0 + P_1 P_0 C_0$$

...

Ritardo **logaritmico** nel numero di bit

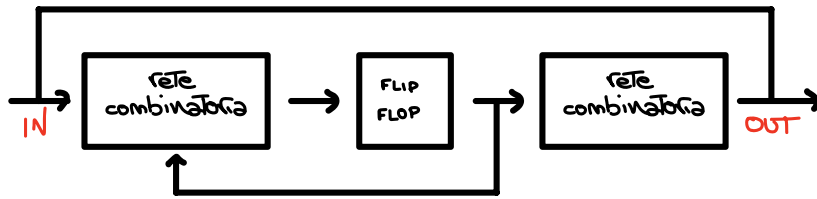
LOGICA COMBINATORIA

Macchina di Moore



L'output dipende solo da stato al tempo corrente

Macchina di Mealy



L'output dipende dallo stato a tempo corrente e input corrente
 ↳ (RESET di vending machine)

Flip Flop; elemento di logica sequenziale che risponde solo allo scatto del clock

latch; risponde sempre quando clock = 1

Flip Flop Tipo D

IN	CK	OUT
1	✓	1
0	✓	0
-	0	Q
-	1	Q

Latch Tipo D

IN	CK	OUT
1	1	1
0	1	0
0	0	Q
1	0	Q

} trasparente
 } opaco

Flip-Flop / Latch con Load; se Load = 0 il clock viene ignorato

Flip Flop / Latch SET-RESET

FF

S	R	CK	OUT
0	0	✓	Q
0	1	✓	0
1	0	✓	1
1	1	✓	N.A
-	-	0	Q
-	-	1	Q

LATCH

S	R	CK	OUT
0	0	1	Q
1	0	1	1
0	1	1	0
1	1	1	N.A
-	-	0	Q
-	-	0	Q

Flip Flop Tipo T

FF

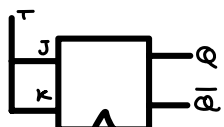
T	CK	OUT
0	✓	Q
1	✓	Q
-	0	Q
-	1	Q

→ inverti
 se T = 1

→ FF JK da
 senza a 11
 mettendo Q

Ottendere Flip-Flop da JKN Flip Flop

T da JK



D da JK

